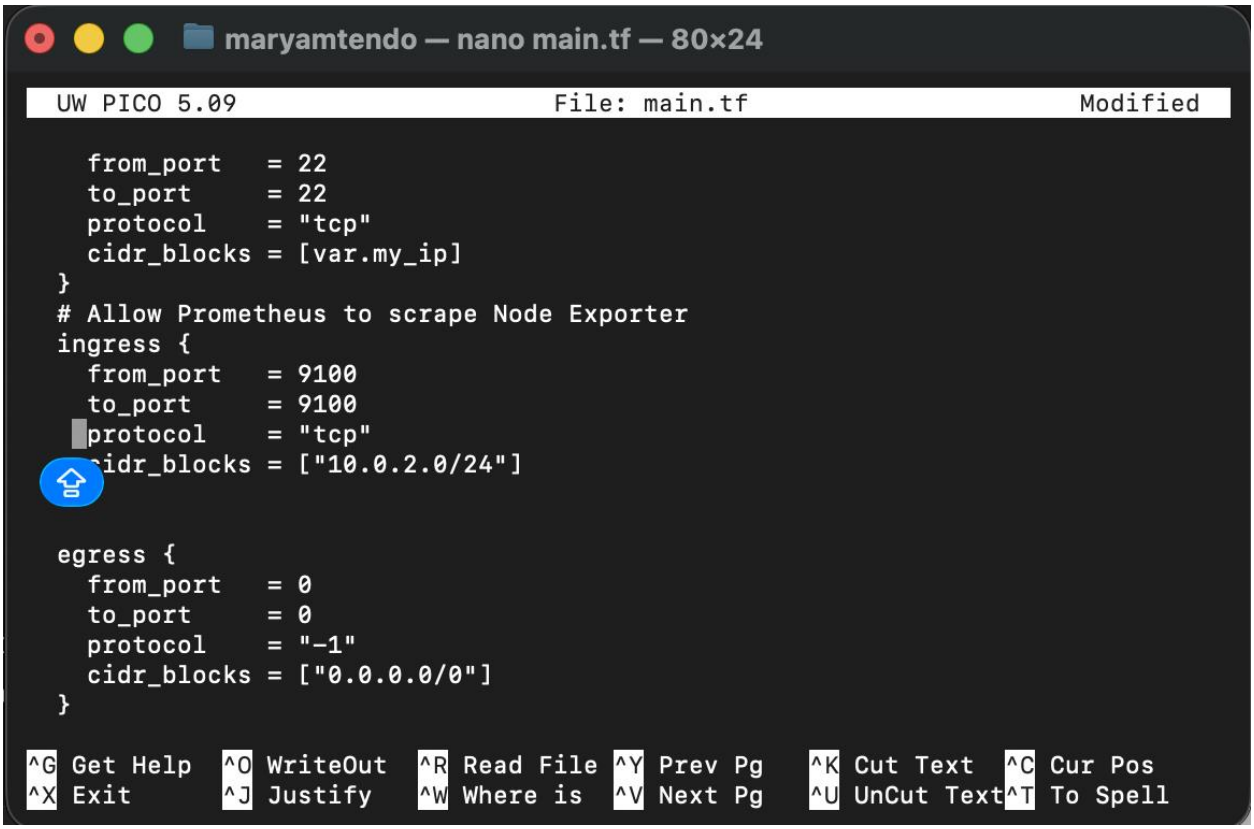
1. I updated the Packer template to install Node Exporter on every EC2 instance which is a small program that runs on each server and exposes system metrics like CPU and memory so prometheus can collect them.
2. Then I configured my AWS credentials from the lab session using the AWS CLI
3. I then ran packer build to create a new custom AMI that now includes Docker, the SSH public key, and Node Exporter then I updated the new ami id and updated it in terraform.tfvars.

```
maryamtendo — bash — 80×24

==> amazon-ebs.amazon-linux: Stopping the source instance...
==> amazon-ebs.amazon-linux: Stopping instance
==> amazon-ebs.amazon-linux: Waiting for the instance to stop...
==> amazon-ebs.amazon-linux: Creating AMI devops-assignment8-1774950169 from ins
tance i-0ed63672d9172cb29
==> amazon-ebs.amazon-linux: Attaching run tags to AMI...
==> amazon-ebs.amazon-linux: AMI: ami-0a4ac2b65a9e38f80
==> amazon-ebs.amazon-linux: Waiting for AMI to become ready...
==> amazon-ebs.amazon-linux: Skipping Enable AMI deprecation...
==> amazon-ebs.amazon-linux: Skipping Enable AMI deregistration protection...
==> amazon-ebs.amazon-linux: Terminating the source AWS instance...
==> amazon-ebs.amazon-linux: Cleaning up any extra volumes...
==> amazon-ebs.amazon-linux: No volumes to clean up, skipping
==> amazon-ebs.amazon-linux: Deleting temporary security group...
==> amazon-ebs.amazon-linux: Deleting temporary keypair...
Build 'amazon-ebs.amazon-linux' finished after 5 minutes 19 seconds.

==> Wait completed after 5 minutes 19 seconds

==> Builds finished. The artifacts of successful builds are:
--> amazon-ebs.amazon-linux: AMIs were created:
us-east-1: ami-0a4ac2b65a9e38f80

bash-3.2$
```

4. After, I created a new Terraform file called monitoring.tf to provision a new EC2 instance in the private subnet that runs both prometheus and grafana. It also creates a dedicated security group for the monitoring server.
5. Then I updated the private instance security group to allow traffic on port 9100. So that prometheus can reach the node exporter running on each private EC2 and collect its metrics.



6. I then, run terraform apply was run to provision all 26 AWS resources including the VPC, subnets, bastion host, 6 private instances, and the new monitoring server running prometheus and grafana.

```
maryamtendo — bash — 80×24
module.vpc.aws_nat_gateway.this[0]: Still creating... [01m00s elapsed]
module.vpc.aws_nat_gateway.this[0]: Still creating... [01m10s elapsed]
module.vpc.aws_nat_gateway.this[0]: Creation complete after 1m17s [id=nat-0dc3ad
225a5bcbb08]
module.vpc.aws_route.private_nat_gateway[0]: Creating...
module.vpc.aws_route.private_nat_gateway[0]: Creation complete after 2s [id=r-rt
b-01d83d189c227ab1f1080289494]

Apply complete! Resources: 26 added, 0 changed, 0 destroyed.

Outputs:

bastion_public_ip = "3.91.59.33"
monitoring_private_ip = "10.0.2.164"
private_instance_ips = [
  "10.0.2.54",
  "10.0.2.228",
  "10.0.2.124",
  "10.0.2.64",
  "10.0.2.206",
  "10.0.2.128",
]
vpc_id = "vpc-064018e40d2bc394b"
bash-3.2$
```

7. I updated the prometheus config file with the new private Ip addresses of the 6 EC2 instances because the ips changed since the infrastructure was rebuilt.

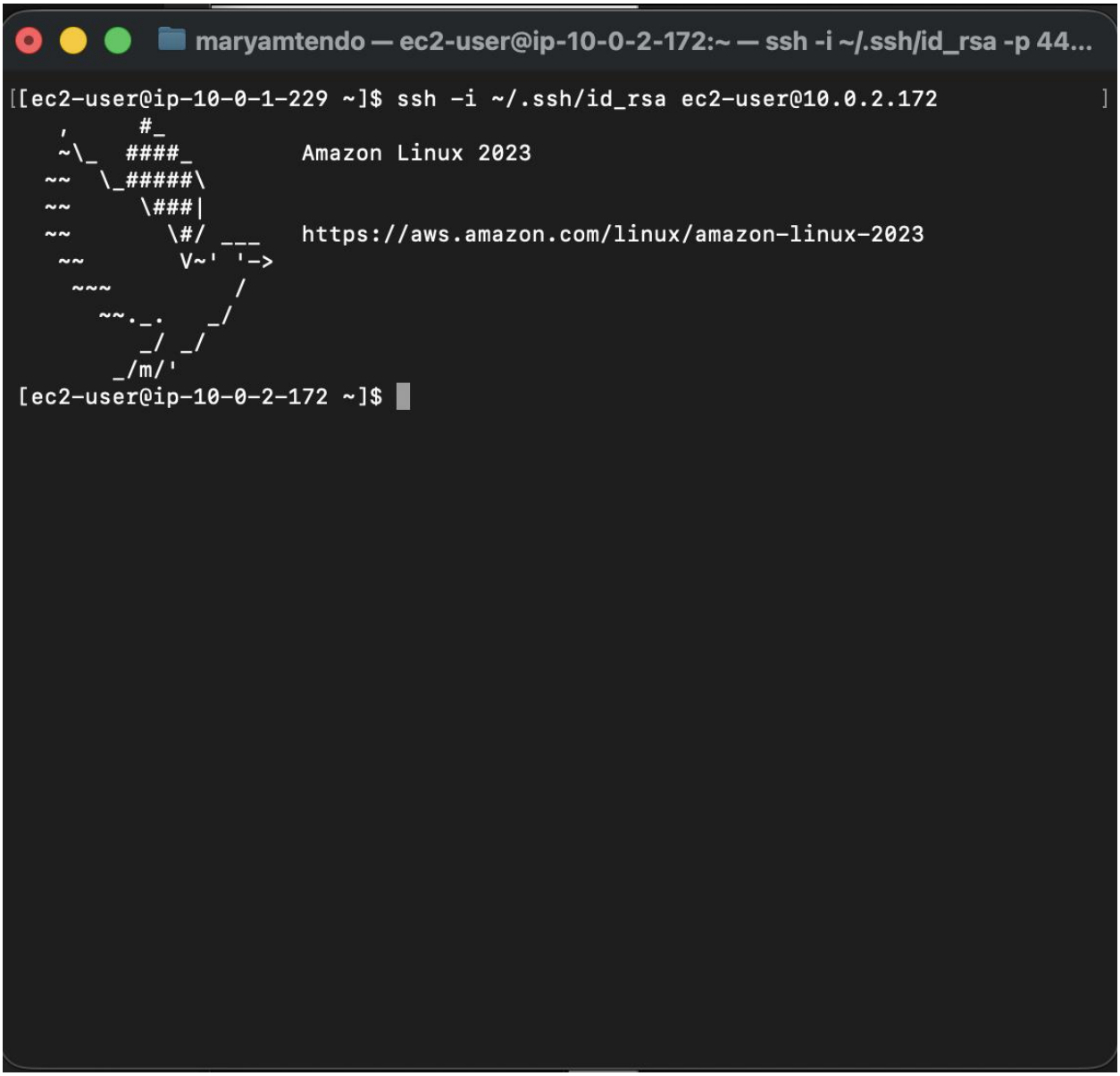
```
maryamtendo — nano monitoring.tf — 80×24
UW PICO 5.09                    File: monitoring.tf                    Modified

    # Prometheus config - scrape all private instances
    cat > /etc/prometheus/prometheus.yml << 'PROMEOF'
    global:
      scrape_interval: 15s
    scrape_configs:
      - job_name: 'node_exporter'
        static_configs:
          - targets:
            - '10.0.2.54:9100'
            - '10.0.2.228:9100'
            - '10.0.2.124:9100'
            - '10.0.2.64:9100'
            - '10.0.2.206:9100'
            - '10.0.2.128:9100'
    PROMEOF

    # Prometheus service
    cat > /etc/systemd/system/prometheus.service << 'SVCEOF'
    [Unit]

^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Pg   ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is  ^V Next Pg   ^U UnCut Text^T To Spell
```

8. Then I confirmed SSH into the bastion host to be working since the bastion is in the public subnet, it acts as the entry point into the private network. I configured it on port 443 because of restrictions I encountered at first with 22.

maryamtendo — ec2-user@ip-10-0-1-229:~ — ssh -i ~/.ssh/id_rsa -p 44...

```
Apply complete! Resources: 26 added, 0 changed, 0 destroyed.

Outputs:

bastion_public_ip = "44.220.164.244"
monitoring_private_ip = "10.0.2.172"
private_instance_ips = [
  "10.0.2.23",
  "10.0.2.6",
  "10.0.2.254",
  "10.0.2.44",
  "10.0.2.149",
  "10.0.2.187",
]
vpc_id = "vpc-028fd5c8ecf6e6fff"
bash-3.2$ ssh -i ~/.ssh/id_rsa -p 443 ec2-user@44.220.164.244
The authenticity of host '[44.220.164.244]:443 ([44.220.164.244]:443)' can't be
established.
ED25519 key fingerprint is SHA256:HFfLjRI+2nD5R0Qz0Y6Zox8AkVHFebQJFrajsEBIZX8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[44.220.164.244]:443' (ED25519) to the list of known
 hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-1-229 ~]$
```

9. From the bastion, I SSH into the monitoring server in the private subnet. This is the server running prometheus and grafana.



10. Prometheus was running successfully on the monitoring server. It set it up to be installed automatically via the user_data script so when the EC2 was launched, it started.

```
maryamtendo — ec2-user@ip-10-0-2-172:~ — ssh -i ~/.ssh/id_rsa -p 44...
[[ec2-user@ip-10-0-1-229 ~]$ ssh -i ~/.ssh/id_rsa ec2-user@10.0.2.172
       ,     #_
       ~\_  ####_        Amazon Linux 2023
      ~~  \_#####\
      ~~     \###|
      ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
       ~~       V~' '->
        ~~~         /
          ~~._.   _/
             _/ _/
           _/m/'
[[ec2-user@ip-10-0-2-172 ~]$ sudo systemctl status prometheus
● prometheus.service - Prometheus
     Loaded: loaded (/etc/systemd/system/prometheus.service; enabled; preset: d>
     Active: active (running) since Wed 2026-04-01 07:28:33 UTC; 28min ago
   Main PID: 2808 (prometheus)
      Tasks: 6 (limit: 1096)
     Memory: 70.2M
        CPU: 900ms
     CGroup: /system.slice/prometheus.service
             └─2808 /usr/local/bin/prometheus --config.file=/etc/prometheus/pro>

Apr 01 07:28:37 ip-10-0-2-172.ec2.internal prometheus[2808]: ts=2026-04-01T07:2>
Apr 01 07:28:37 ip-10-0-2-172.ec2.internal prometheus[2808]: ts=2026-04-01T07:2>
Apr 01 07:28:37 ip-10-0-2-172.ec2.internal prometheus[2808]: ts=2026-04-01T07:2>
Apr 01 07:28:38 ip-10-0-2-172.ec2.internal prometheus[2808]: ts=2026-04-01T07:2>
Apr 01 07:28:38 ip-10-0-2-172.ec2.internal prometheus[2808]: ts=2026-04-01T07:2>
Apr 01 07:28:38 ip-10-0-2-172.ec2.internal prometheus[2808]: ts=2026-04-01T07:2>
Apr 01 07:28:38 ip-10-0-2-172.ec2.internal prometheus[2808]: ts=2026-04-01T07:2>
Apr 01 07:28:38 ip-10-0-2-172.ec2.internal prometheus[2808]: ts=2026-04-01T07:2>
Apr 01 07:28:38 ip-10-0-2-172.ec2.internal prometheus[2808]: ts=2026-04-01T07:2>
Apr 01 07:28:38 ip-10-0-2-172.ec2.internal prometheus[2808]: ts=2026-04-01T07:2>
lines 1-20/20 (END)
```

11. Ithen installed grafana which initially failed because the EC2 disk was 99% full. When I checked, the default EC2 volume is only 2GB and prometheus had used most of it so I updated the terraform config to increase the monitoring server disk to 20GB.

maryamtendo — ec2-user@ip-10-0-2-172:~ — ssh -i ~/.ssh/id_rsa -p 44...

```
baseurl=https://rpm.grafana.com
repo_gpgcheck=1
enabled=1
gpgcheck=1
gpgkey=https://rpm.grafana.com/gpg.key
sslverify=1
sslcacert=/etc/pki/tls/certs/ca-bundle.crt
EOF'
[ec2-user@ip-10-0-2-172 ~]$ sudo dnf install -y grafana
grafana                                          87 MB/s |  35 MB     00:00
Errors during downloading metadata for repository 'grafana':
  - Curl error (23): Failed writing received data to disk/application for https:
//rpm.grafana.com/repodata/1b963cd336683eeb343ca03a1bc534fe2e60d19f140e0638be46a
aeb831ab949-filelists.xml.gz [Failure writing output to destination, passed 1635
7 returned 3328]
Error: Failed to download metadata for repo 'grafana': Yum repo downloading erro
r: Downloading error(s): repodata/1b963cd336683eeb343ca03a1bc534fe2e60d19f140e06
38be46aaeb831ab949-filelists.xml.gz - Download failed: Curl error (23): Failed w
riting received data to disk/application for https://rpm.grafana.com/repodata/1b
963cd336683eeb343ca03a1bc534fe2e60d19f140e0638be46aaeb831ab949-filelists.xml.gz
[Failure writing output to destination, passed 16357 returned 3328]
Ignoring repositories: grafana
Last metadata expiration check: 22:17:12 ago on Tue Mar 31 09:44:12 2026.
No match for argument: grafana
Error: Unable to find a match: grafana
[ec2-user@ip-10-0-2-172 ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        457M     0  457M   0% /dev
tmpfs           480M     0  480M   0% /dev/shm
tmpfs           192M  2.9M  189M   2% /run
/dev/xvda1      2.0G  1.9G   36M  99% /
tmpfs           480M     0  480M   0% /tmp
/dev/xvda128     10M  1.3M  8.7M  13% /boot/efi
tmpfs            96M     0   96M   0% /run/user/1000
[ec2-user@ip-10-0-2-172 ~]$
```

```
maryamtendo — ec2-user@ip-10-0-2-172:~ — ssh -i ~/.ssh/id_rsa -p 44...

  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Wed Apr  1 07:55:26 2026 from 10.0.1.229
[ec2-user@ip-10-0-2-172 ~]$ sudo growpart /dev/xvda 1
sudo xfs_growfs /
CHANGED: partition=1 start=24576 old: size=4169695 end=4194271 new: size=4191843
1 end=41943007
meta-data=/dev/xvda1             isize=512    agcount=2, agsize=260608 blks
         =                       sectsz=4096  attr=2, projid32bit=1
         =                       crc=1        finobt=1, sparse=1, rmapbt=0
         =                       reflink=1    bigtime=1 inobtcount=1 nrext64=0
         =                       exchange=0
data     =                       bsize=4096   blocks=521211, imaxpct=25
         =                       sunit=128    swidth=128 blks
naming   =version 2              bsize=16384  ascii-ci=0, ftype=1, parent=0
log      =internal log           bsize=4096   blocks=16384, version=2
         =                       sectsz=4096  sunit=4 blks, lazy-count=1
realtime =none                   extsz=4096   blocks=0, rtextents=0
data blocks changed from 521211 to 5239803
[ec2-user@ip-10-0-2-172 ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        457M     0  457M   0% /dev
tmpfs           480M     0  480M   0% /dev/shm
tmpfs           192M  2.9M  189M   2% /run
/dev/xvda1       20G  2.1G   18G  11% /
tmpfs           480M     0  480M   0% /tmp
/dev/xvda128     10M  1.3M  8.7M  13% /boot/efi
tmpfs            96M     0   96M   0% /run/user/1000
[ec2-user@ip-10-0-2-172 ~]$
```

12. Grafana successfully installed and also it starts automatically on boot.

```
maryamtendo — ec2-user@ip-10-0-2-172:~ — ssh -i ~/.ssh/id_rsa -p 44...

grafana_12.4.2_23531306697_linux_amd64.rpm       54 MB/s | 188 MB     00:03
--------------------------------------------------------------------------------
Total                                            54 MB/s | 188 MB     00:03
grafana                                         124 kB/s | 2.4 kB     00:00
Importing GPG key 0x10458545:
 Userid     : "Grafana Labs <engineering@grafana.com>"
 Fingerprint: B53A E77B ADB6 30A6 8304 6005 963F A277 1045 8545
 From       : https://rpm.grafana.com/gpg.key
Key imported successfully
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Installing       : grafana-12.4.2-1.x86_64                                1/1
  Running scriptlet: grafana-12.4.2-1.x86_64                                1/1
### NOT starting on installation, please execute the following statements to con
figure grafana to start automatically using systemd
 sudo /bin/systemctl daemon-reload
 sudo /bin/systemctl enable grafana-server.service
### You can start grafana-server by executing
 sudo /bin/systemctl start grafana-server.service

POSTTRANS: Running script

  Verifying        : grafana-12.4.2-1.x86_64                                1/1

Installed:
  grafana-12.4.2-1.x86_64

Complete!
Created symlink /etc/systemd/system/multi-user.target.wants/grafana-server.servi
ce → /usr/lib/systemd/system/grafana-server.service.
[ec2-user@ip-10-0-2-172 ~]$
```

13. I verified that grafana service status was active and running. It listens on port 3000 and is accessible via an SSH tunnel from the local machine.

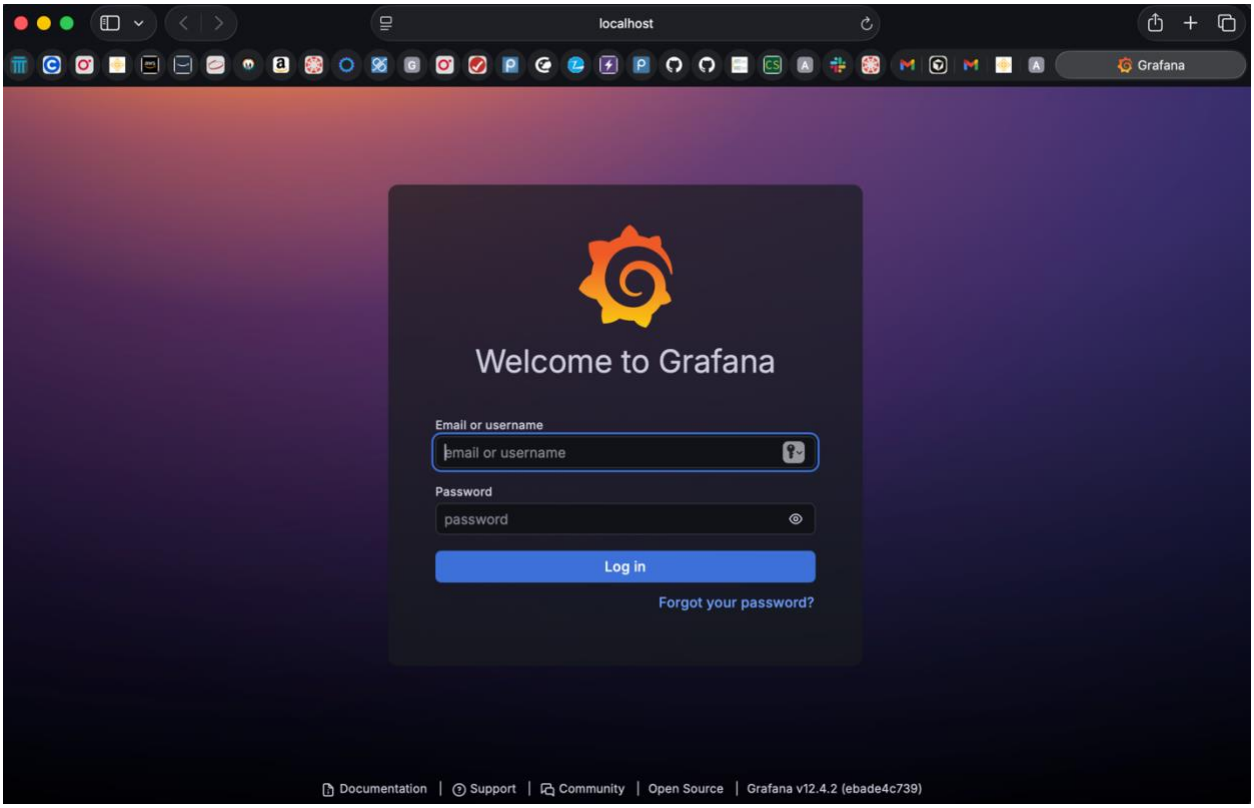
maryamtendo — ec2-user@ip-10-0-2-172:~ — ssh -i ~/.ssh/id_rsa -p 44...

```
 sudo /bin/systemctl start grafana-server.service

POSTTRANS: Running script

  Verifying        : grafana-12.4.2-1.x86_64                                1/1

Installed:
  grafana-12.4.2-1.x86_64

Complete!
Created symlink /etc/systemd/system/multi-user.target.wants/grafana-server.servi
ce → /usr/lib/systemd/system/grafana-server.service.
[[ec2-user@ip-10-0-2-172 ~]$ sudo systemctl status grafana-server
● grafana-server.service - Grafana instance
     Loaded: loaded (/usr/lib/systemd/system/grafana-server.service; enabled; p>
     Active: active (running) since Wed 2026-04-01 08:14:06 UTC; 2min 8s ago
       Docs: http://docs.grafana.org
   Main PID: 4552 (grafana)
      Tasks: 7 (limit: 1096)
     Memory: 333.9M
        CPU: 5.948s
     CGroup: /system.slice/grafana-server.service
             └─4552 /usr/share/grafana/bin/grafana server --config=/etc/grafana>

Apr 01 08:15:07 ip-10-0-2-172.ec2.internal grafana[4552]: logger=dashboard-serv>
Apr 01 08:15:37 ip-10-0-2-172.ec2.internal grafana[4552]: logger=dashboard-serv>
Apr 01 08:15:41 ip-10-0-2-172.ec2.internal grafana[4552]: logger=bleve-backend >
Apr 01 08:15:41 ip-10-0-2-172.ec2.internal grafana[4552]: logger=bleve-backend >
Apr 01 08:15:41 ip-10-0-2-172.ec2.internal grafana[4552]: logger=bleve-backend >
Apr 01 08:15:41 ip-10-0-2-172.ec2.internal grafana[4552]: logger=bleve-backend >
Apr 01 08:15:41 ip-10-0-2-172.ec2.internal grafana[4552]: logger=bleve-backend >
Apr 01 08:15:41 ip-10-0-2-172.ec2.internal grafana[4552]: logger=bleve-backend >
Apr 01 08:15:41 ip-10-0-2-172.ec2.internal grafana[4552]: logger=infra.usagesta>
Apr 01 08:16:07 ip-10-0-2-172.ec2.internal grafana[4552]: logger=dashboard-serv>
lines 1-21/21 (END)
```

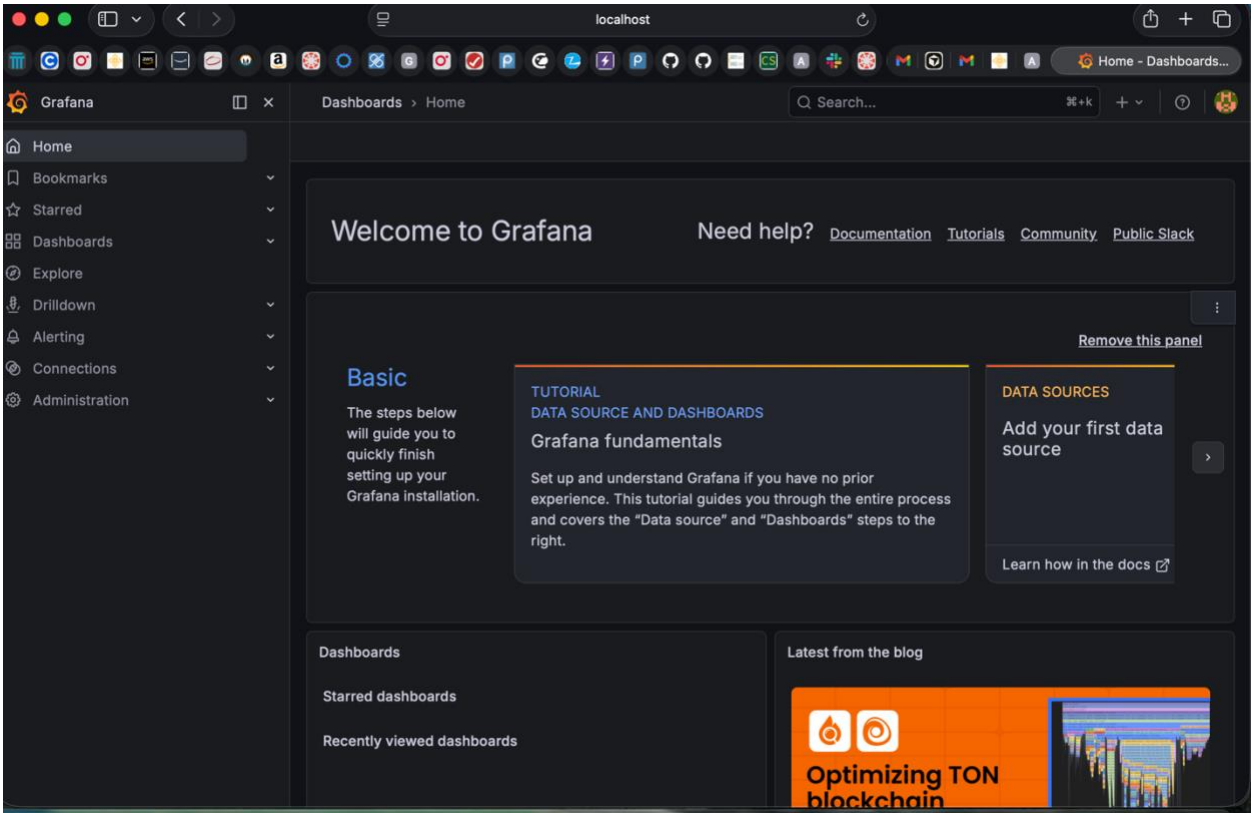
14. I then verified prometheus to be scraping all 6 private EC2 instances on port 9100 via the node exporter.

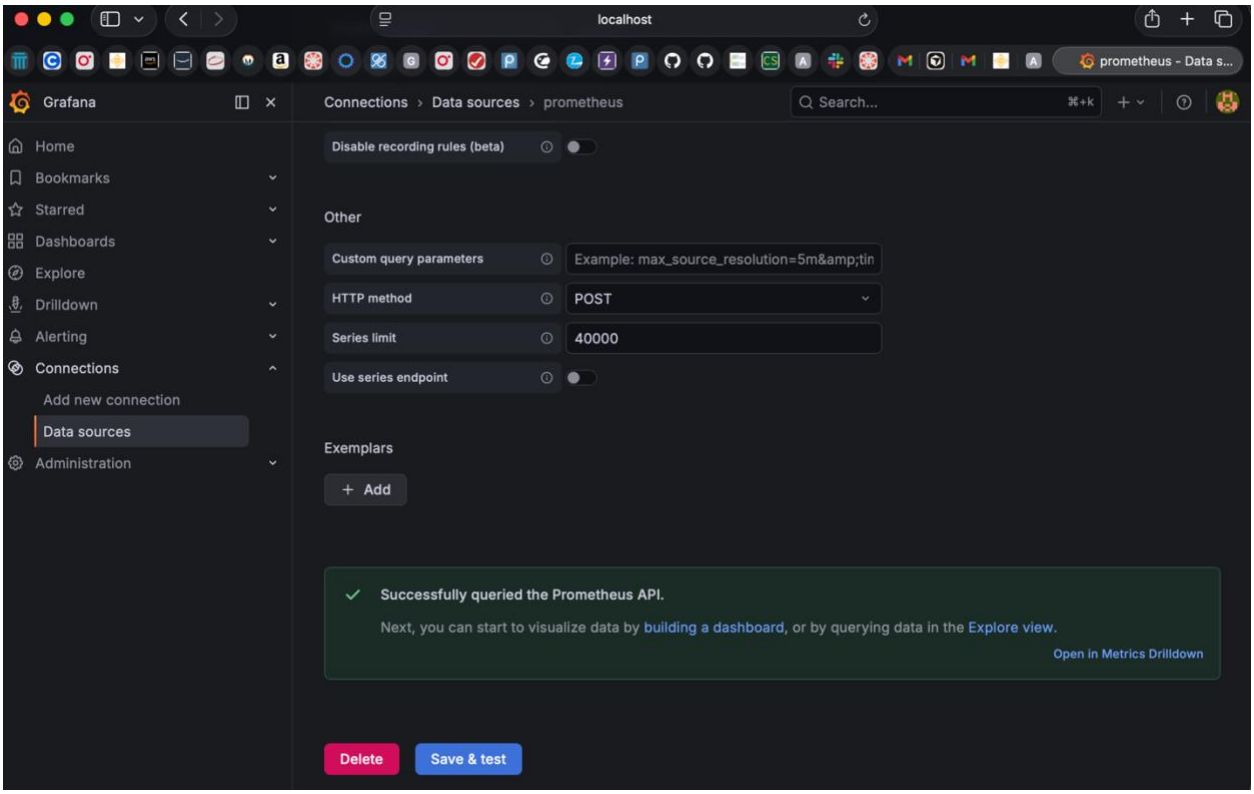
```
maryamtendo — ec2-user@ip-10-0-2-172:~ — ssh -i ~/.ssh/id_rsa -p 44...

[[ec2-user@ip-10-0-2-172 ~]$ curl http://localhost:9090/api/v1/targets | python3 ]
-m json.tool | grep -E "health|instance"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  3115   0  3115   0     0 494680     0  --:--:-- --:--:-- --:--:-- 519166
                    "instance": "10.0.2.23:9100",
                "health": "up",
                    "instance": "10.0.2.6:9100",
                "health": "up",
                    "instance": "10.0.2.254:9100",
                "health": "unknown",
                    "instance": "10.0.2.44:9100",
                "health": "up",
                    "instance": "10.0.2.149:9100",
                "health": "up",
                    "instance": "10.0.2.187:9100",
                "health": "unknown",
[ec2-user@ip-10-0-2-172 ~]$
```

15. After, I created SSH tunnel to forward local port 3000 through the bastion to the monitoring server and this allowed Grafana to be accessed in the browser at http://localhost:3000 even though it lives in a private subnet.

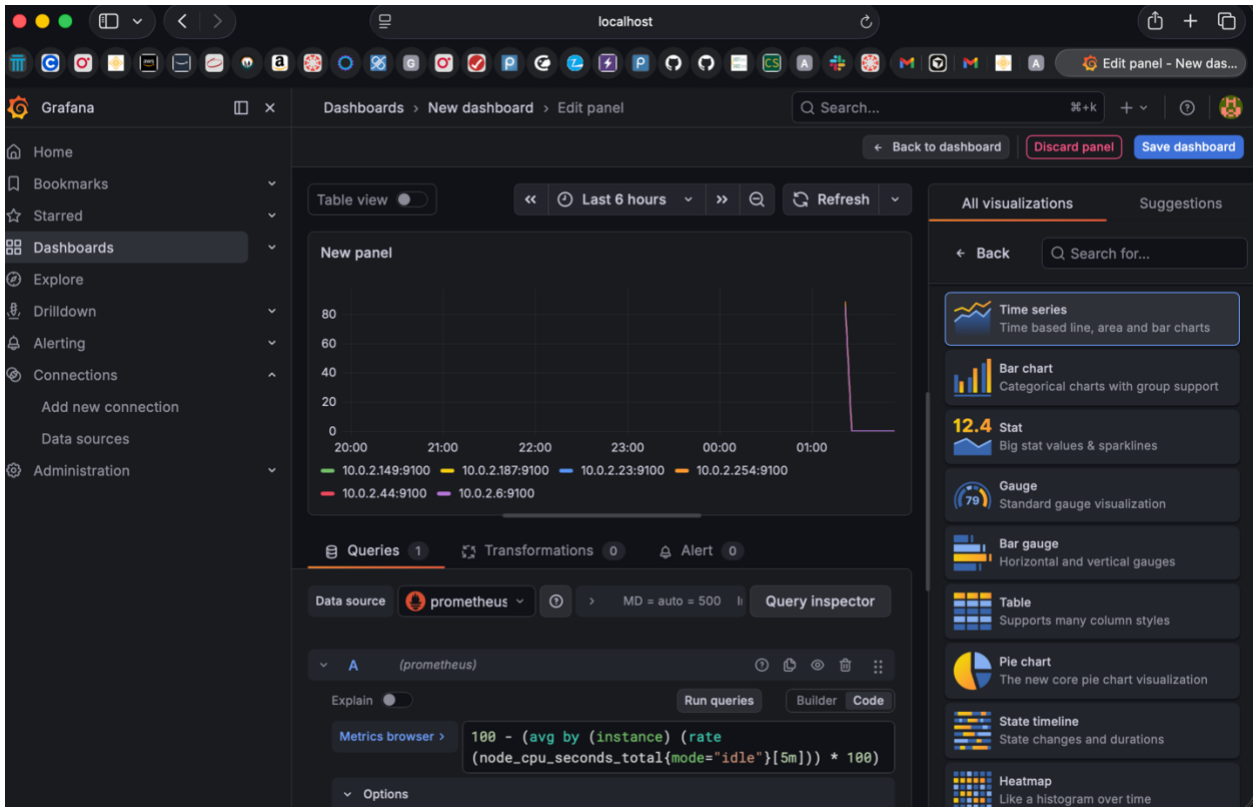


Logged into Grafana using the default credentials (admin/admin). Added Prometheus was as a data source using the URL http://localhost:9090. Grafana confirmed it could successfully query the Prometheus API.

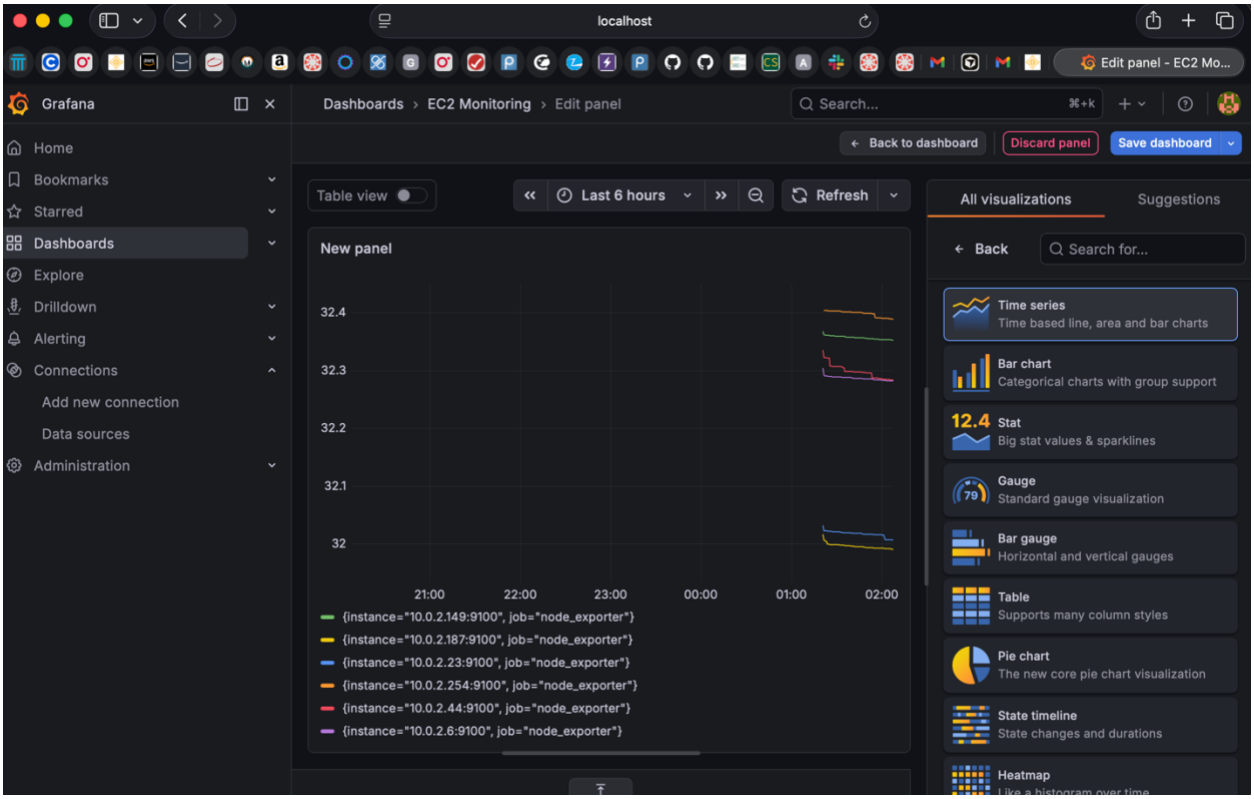

localhost
Home - Dashboards...
Grafana
Dashboards › Home
Search...
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Administration
Welcome to Grafana
Need help?
Documentation
Tutorials
Community
Public Slack
Remove this panel
Basic
The steps below will guide you to quickly finish setting up your Grafana installation.
TUTORIAL
DATA SOURCE AND DASHBOARDS
Grafana fundamentals
Set up and understand Grafana if you have no prior experience. This tutorial guides you through the entire process and covers the "Data source" and "Dashboards" steps to the right.
DATA SOURCES
Add your first data source
Learn how in the docs
Dashboards
Starred dashboards
Recently viewed dashboards
Latest from the blog
Optimizing TON blockchain

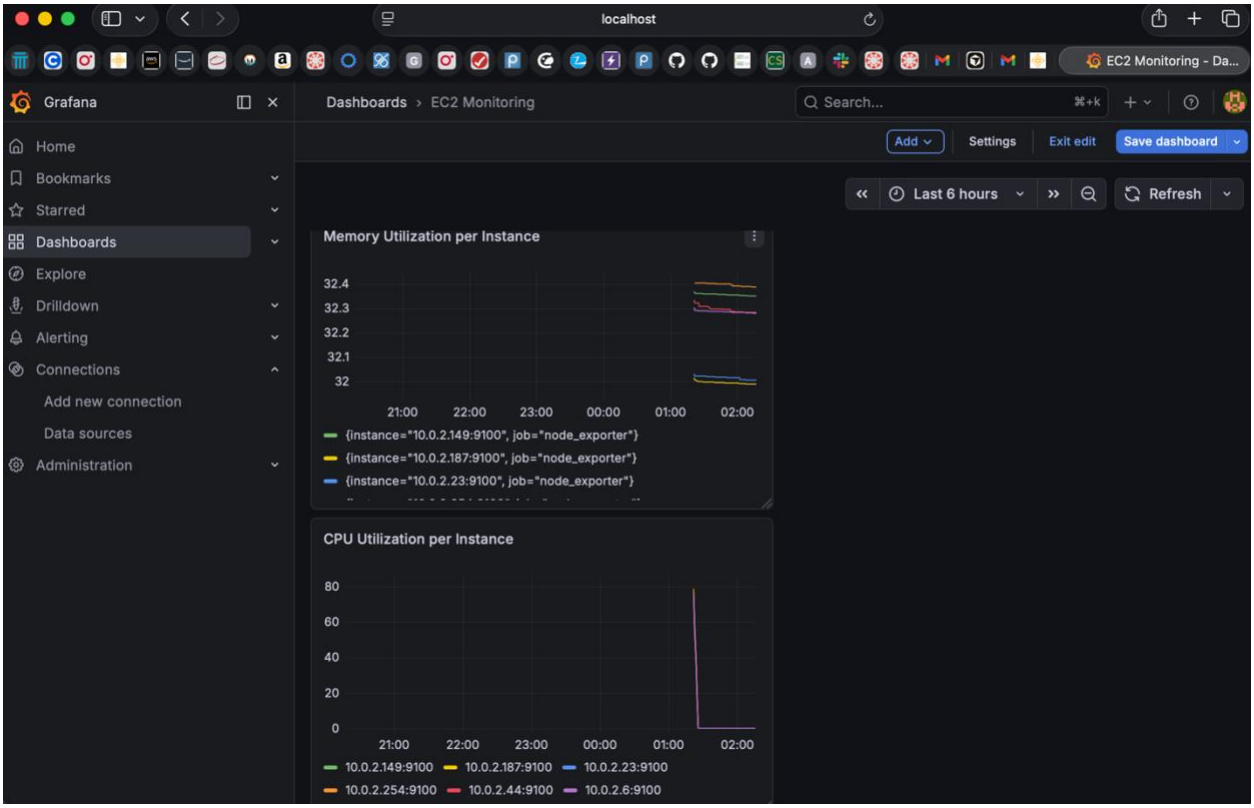

localhost
prometheus - Data s...
Grafana
Connections › Data sources › prometheus
Search...
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Add new connection
Data sources
Administration
Disable recording rules (beta)
Other
Custom query parameters
Example: max_source_resolution=5m&tin
HTTP method
POST
Series limit
40000
Use series endpoint
Exemplars
+ Add
Successfully queried the Prometheus API.
Next, you can start to visualize data by building a dashboard, or by querying data in the Explore view.
Open in Metrics Drilldown
Delete
Save & test

16. I then for bonus created a dashboard to monitor ec2 and with a CPU Utilization panel. The query calculates the percentage of CPU being used across all 6 private instances and displays them as a time series graph.



17. I created a second panel showing Memory Utilization per Instance. The query calculates the percentage of memory being used on each EC2 instance in real time. Both panels together make up the EC2 Monitoring dashboard.


localhost
Edit panel - EC2 Mo...
Grafana
Dashboards › EC2 Monitoring › Edit panel
Search...
Back to dashboard
Discard panel
Save dashboard
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Add new connection
Data sources
Administration
Table view
Last 6 hours
Refresh
All visualizations
Suggestions
New panel
Back
Search for...
32.4
32.3
32.2
32.1
32
21:00
22:00
23:00
00:00
01:00
02:00
{instance="10.0.2.149:9100", job="node_exporter"}
{instance="10.0.2.187:9100", job="node_exporter"}
{instance="10.0.2.23:9100", job="node_exporter"}
{instance="10.0.2.254:9100", job="node_exporter"}
{instance="10.0.2.44:9100", job="node_exporter"}
{instance="10.0.2.6:9100", job="node_exporter"}
Time series
Time based line, area and bar charts
Bar chart
Categorical charts with group support
12.4
Stat
Big stat values & sparklines
Gauge
Standard gauge visualization
Bar gauge
Horizontal and vertical gauges
Table
Supports many column styles
Pie chart
The new core pie chart visualization
State timeline
State changes and durations
Heatmap


localhost
EC2 Monitoring - Da...
Grafana
Dashboards › EC2 Monitoring
Search...
Add
Settings
Exit edit
Save dashboard
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Add new connection
Data sources
Administration
Last 6 hours
Refresh
Memory Utilization per Instance
32.4
32.3
32.2
32.1
32
21:00
22:00
23:00
00:00
01:00
02:00
{instance="10.0.2.149:9100", job="node_exporter"}
{instance="10.0.2.187:9100", job="node_exporter"}
{instance="10.0.2.23:9100", job="node_exporter"}
CPU Utilization per Instance
80
60
40
20
0
21:00
22:00
23:00
00:00
01:00
02:00
10.0.2.149:9100
10.0.2.187:9100
10.0.2.23:9100
10.0.2.254:9100
10.0.2.44:9100
10.0.2.6:9100

How to Run

- Have Packer and Terraform installed via Homebrew. AWS CLI configured. SSH key. Update the ip address in terraform.tfvars before running.
- After, go to the packer folder and run, packer init amazon-linux.pkr.hcl, then packer validate amazon-linux.pkr.hcl, then packer build amazon-linux.pkr.hcl. Copy the ami id you see from the output and paste it into terraform/terraform.tfvars as the ami_id value.
- Then go to the terraform folder and run, terraform init, then terraform apply. Type yes when prompted. This creates the VPC, subnets, bastion host, 6 private EC2 instances, and the monitoring server.
- After terraform apply completes you expect to see the bastion public IP, monitoring server private IP, and the private IPs of all 6 EC2 instances printed as outputs. A total of 26 AWS resources will be created including the VPC, subnets, security groups, NAT gateway, internet gateway, and all EC2 instances.
- SSH uses port 443 instead of 22 to avoid network restrictions. SSH into bastion and from the bastion you can SSH into any of the private instances.
- To destroy all AWS resources and stop charges, run terraform destroy from the terraform folder and type yes when prompted.